ResQnet

Database, API & API-Key Architecture

Read-only technical architecture extraction from the ResQnet repository reconnaissance.

1. Database

DB: PostgreSQL 16 + PostGIS 3.4 **Runtime:** asyncpg **Migrations:** psycopg2

Alembic head: 71867cddef4b **Spatial reference:** SRID 4326 (WGS84)

#	Table	Purpose
1	users	Authentication and user identity
2	intersections	Road graph nodes
3	roads	Road graph edges / road state
4	hazard_zones	Spatial hazards
5	incident_reports	Citizen incident signals
6	report_clusters	Spatial/temporal clusters
7	responders	Responder profiles/status
8	missions	Operational assignments
9	field_surveys	Field verification/evidence
10	safety_hubs	Evacuation/support locations and capacity
11	hub_resources	Hub resources
12	resource_requirements	Resource shortfalls
13	campaigns	Relief campaigns
14	donations	Campaign donations
15	emergency_alerts	Emergency alerts/affected areas
16	sync_queue_log	Offline sync/replay records
17	audit_logs	Audit trail

Major relationships

```
Graph: intersections (nodes) ↔ roads (edges) ↔ safety_hubs

Operations: incident_reports → report_clusters → missions → field_surveys → roads / hazard_zones

Relief: safety_hubs ↔ hub_resources ↔ resource_requirements ↔ campaigns ↔ donations
```

PostGIS uses POINT, LINESTRING, POLYGON and MULTIPOLYGON geometries with SRID 4326 and spatial GiST indexing.

2. API Architecture

Base: /api/v1 **Compatibility:** /api mounts also exist.

Framework: FastAPI modular monolith

Auth: OAuth2 Bearer + JWT (HS256) + bcrypt password hashing

Roles: citizen, responder, coordinator, fundraiser

Domain routers: auth, incident-reports, clusters, missions, field_surveys, hazards, routing, hubs, resources, campaigns, donations, alerts, roads, sync, simulation.

```
Frontend → API Client → /api/v1 FastAPI Router → Domain Service/Business Logic → SQLAlchemy → PostgreSQL/PostGIS
```

Routing additionally uses the in-process Dijkstra engine with road, hazard and Safety Hub data.

3. API-Key & Configuration Structure

External service identified: OpenStreetMap raster tiles through MapLibre GL JS. No proprietary map API key was identified.

Security note: actual secret values are intentionally excluded.

Variable	Purpose	Scope	Secret
DATABASE_URL	Database connection	Backend	Yes
SYNC_DATABASE_URL	Sync/database connection	Backend	Yes
SECRET_KEY / JWT_SECRET_KEY	JWT/application signing	Backend	Yes
POSTGRES_DB	Database name	Runtime	No
POSTGRES_USER	Database username	Runtime	Credential
POSTGRES_PASSWORD	Database password	Runtime	Yes

Frontend exposure

No VITE_/import.meta.env API-key variables were identified. Frontend calls use relative /api/v1 paths through the application/proxy configuration.

High-level architecture

```
React + TypeScript + Vite + Tailwind + MapLibre
↓
/api/v1
↓
FastAPI modular monolith
↓
SQLAlchemy
↓
PostgreSQL 16 + PostGIS 3.4
```

Summary

Database: 17 application tables; Alembic head 71867cddef4b. API: FastAPI under /api/v1. Authentication: OAuth2 Bearer/JWT HS256 with bcrypt. Roles: citizen, responder, coordinator, fundraiser. External map service: OpenStreetMap. Secret values are not included.